

```

# LeNet on CIFAR-10
# -----

import time
import numpy as np
from tqdm import tqdm
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
import tensorflow as tf
from tensorflow.keras import layers, models, datasets, utils

# ---- Build LeNet ----
def build_lenet(input_shape=(28, 28, 1), num_classes=10):
    model = models.Sequential([
        layers.Conv2D(6, (5, 5), activation='tanh', padding='same',
input_shape=input_shape),
        layers.AveragePooling2D(pool_size=(2, 2)),
        layers.Conv2D(16, (5, 5), activation='tanh'),
        layers.AveragePooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dense(120, activation='tanh'),
        layers.Dense(84, activation='tanh'),
        layers.Dense(num_classes, activation='softmax')
    ])
    return model

# ---- Load MNIST ----
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0
y_train = utils.to_categorical(y_train)
y_test = utils.to_categorical(y_test)

# ---- Train ----
model = build_lenet()
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

start = time.time()
for epoch in tqdm(range(10), desc="Training LeNet"):
    model.fit(x_train, y_train, epochs=10, batch_size=32, verbose=0)
    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    print(f"Epoch {epoch+1}: Accuracy={acc:.4f}, Loss={loss:.4f}")

# ---- Evaluate ----

```

```

y_pred = np.argmax(model.predict(x_test), axis=1)
y_true = np.argmax(y_test, axis=1)
print("\n✅ Final Results (LeNet):")
print(f"Accuracy: {accuracy_score(y_true, y_pred):.4f}")
print(f"Precision: {precision_score(y_true, y_pred,
average='macro'):.4f}")
print(f"Recall: {recall_score(y_true, y_pred, average='macro'):.4f}")
print(f"F1: {f1_score(y_true, y_pred, average='macro'):.4f}")
print(f"Training time: {time.time() - start:.2f} sec")

model.save("lenet_model.keras")
print("💾 Model saved as lenet_model.keras")

```

```

print("💾 Model saved as lenet_model.keras")

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Training LeNet: 10% | 1/10 [00:57<08:39, 57.75s/it]Epoch 1: Accuracy=0.9852, Loss=0.0498
Training LeNet: 20% | 2/10 [01:45<06:53, 51.65s/it]Epoch 2: Accuracy=0.9847, Loss=0.0603
Training LeNet: 30% | 3/10 [02:31<05:43, 49.11s/it]Epoch 3: Accuracy=0.9877, Loss=0.0562
Training LeNet: 40% | 4/10 [03:17<04:48, 48.00s/it]Epoch 4: Accuracy=0.9867, Loss=0.0567
Training LeNet: 50% | 5/10 [04:04<03:58, 47.71s/it]Epoch 5: Accuracy=0.9868, Loss=0.0625
Training LeNet: 60% | 6/10 [04:51<03:09, 47.40s/it]Epoch 6: Accuracy=0.9880, Loss=0.0658
Training LeNet: 70% | 7/10 [05:38<02:22, 47.42s/it]Epoch 7: Accuracy=0.9873, Loss=0.0653
Training LeNet: 80% | 8/10 [06:24<01:33, 46.96s/it]Epoch 8: Accuracy=0.9875, Loss=0.0739
Training LeNet: 90% | 9/10 [07:11<00:46, 46.71s/it]Epoch 9: Accuracy=0.9868, Loss=0.0710
Training LeNet: 100% | 10/10 [07:57<00:00, 47.74s/it]Epoch 10: Accuracy=0.9876, Loss=0.0752

313/313 ————— 1s 2ms/step

✅ Final Results (LeNet):
Accuracy: 0.9876
Precision: 0.9876
Recall: 0.9875
F1: 0.9875
Training time: 478.73 sec
💾 Model saved as lenet_model.keras

```

```

# AlexNet on CIFAR-10
# -----

import time
import numpy as np
from tqdm import tqdm
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
import tensorflow as tf
from tensorflow.keras import layers, models, datasets, utils

# ---- Build AlexNet ----
def build_alexnet(input_shape=(32, 32, 3), num_classes=10):
    model = models.Sequential([

```

```

        layers.Conv2D(96, (3, 3), strides=1, activation='relu',
input_shape=input_shape), # Adjusted kernel and strides
        layers.MaxPooling2D((3, 3), strides=2),
        layers.Conv2D(256, (5, 5), padding='same', activation='relu'),
        layers.MaxPooling2D((3, 3), strides=2),
        layers.Conv2D(384, (3, 3), padding='same', activation='relu'),
        layers.Conv2D(384, (3, 3), padding='same', activation='relu'),
        layers.Conv2D(256, (3, 3), padding='same', activation='relu'),
        layers.MaxPooling2D((3, 3), strides=2),
        layers.Flatten(),
        layers.Dense(4096, activation='relu'),
        layers.Dense(4096, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])
    return model

# ---- Load CIFAR-10 ----
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
y_train = utils.to_categorical(y_train.flatten())
y_test = utils.to_categorical(y_test.flatten())

# ---- Train ----
model = build_alexnet()
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

start = time.time()
for epoch in tqdm(range(5), desc="Training AlexNet"):
    model.fit(x_train, y_train, epochs=1, batch_size=64, verbose=0) #
Changed epochs to 1
    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    print(f"Epoch {epoch+1}: Accuracy={acc:.4f}, Loss={loss:.4f}")

# ---- Evaluate ----
y_pred = np.argmax(model.predict(x_test), axis=1)
y_true = np.argmax(y_test, axis=1)
print("\n✓ Final Results (AlexNet):")
print(f"Accuracy: {accuracy_score(y_true, y_pred):.4f}")
print(f"Precision: {precision_score(y_true, y_pred,
average='macro'):.4f}")
print(f"Recall: {recall_score(y_true, y_pred, average='macro'):.4f}")
print(f"F1: {f1_score(y_true, y_pred, average='macro'):.4f}")
print(f"Training time: {time.time() - start:.2f} sec")

```

```
model.save("alexnet_model.keras")
print("📁 Model saved as alexnet_model.keras")
```

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass a
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Training AlexNet: 20%|██████| 1/5 [00:38<02:35, 38.81s/it]Epoch 1: Accuracy=0.4960, Loss=1.4018
Training AlexNet: 40%|███████| 2/5 [01:03<01:31, 30.49s/it]Epoch 2: Accuracy=0.5836, Loss=1.1575
Training AlexNet: 60%|████████| 3/5 [01:28<00:55, 27.81s/it]Epoch 3: Accuracy=0.6474, Loss=1.0102
Training AlexNet: 80%|█████████| 4/5 [01:52<00:26, 26.60s/it]Epoch 4: Accuracy=0.6763, Loss=0.9480
Training AlexNet: 100%|██████████| 5/5 [02:17<00:00, 27.51s/it]Epoch 5: Accuracy=0.6997, Loss=0.8841

313/313 ————— 3s 8ms/step

✅ Final Results (AlexNet):
Accuracy: 0.6997
Precision: 0.7066
Recall: 0.6997
F1: 0.6949
Training time: 140.86 sec
📁 Model saved as alexnet_model.keras
```

```
# VGGNet on CIFAR-10
# -----
import time
import numpy as np
from tqdm import tqdm
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
import tensorflow as tf
from tensorflow.keras import layers, models, datasets, utils

# ---- Build VGG ----
def build_vgg(input_shape=(32, 32, 3), num_classes=10):
    model = models.Sequential([
        layers.Conv2D(64, (3, 3), activation='relu', padding='same',
input_shape=input_shape),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
```

```

        layers.Flatten(),
        layers.Dense(512, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])
    return model

# ---- Load CIFAR-10 ----
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
y_train = utils.to_categorical(y_train.flatten())
y_test = utils.to_categorical(y_test.flatten())

# ---- Train ----
model = build_vgg()
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

start = time.time()
for epoch in tqdm(range(3), desc="Training VGGNet"):
    model.fit(x_train, y_train, epochs=3, batch_size=32, verbose=0)
    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    print(f"Epoch {epoch+1}: Accuracy={acc:.4f}, Loss={loss:.4f}")

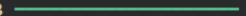
# ---- Evaluate ----
y_pred = np.argmax(model.predict(x_test), axis=1)
y_true = np.argmax(y_test, axis=1)
print("\n✓ Final Results (VGGNet):")
print(f"Accuracy: {accuracy_score(y_true, y_pred):.4f}")
print(f"Precision: {precision_score(y_true, y_pred,
average='macro'):.4f}")
print(f"Recall: {recall_score(y_true, y_pred, average='macro'):.4f}")
print(f"F1: {f1_score(y_true, y_pred, average='macro'):.4f}")
print(f"Training time: {time.time() - start:.2f} sec")

model.save("vgg_model.keras")
print("📁 Model saved as vgg_model.keras")

```

```
model.save("vgg_model.keras")
print("📁 Model saved as vgg_model.keras")
```

```
Training VGGNet: 33%|███████| 1/3 [00:52<01:45, 52.55s/it]Epoch 1: Accuracy=0.7104, Loss=0.8435
Training VGGNet: 67%|███████████| 2/3 [01:36<00:47, 47.36s/it]Epoch 2: Accuracy=0.7436, Loss=0.8265
Training VGGNet: 100%|██████████████████| 3/3 [02:20<00:00, 46.71s/it]Epoch 3: Accuracy=0.7539, Loss=1.0029
```

313/313  2s 4ms/step

```
✅ Final Results (VGGNet):
Accuracy: 0.7539
Precision: 0.7567
Recall: 0.7539
F1: 0.7536
Training time: 142.34 sec
📁 Model saved as vgg_model.keras
```

 Start coding or [generate](#) with AI.